

Leaper

Memory, search, and the road to a moving target

A plain-language explainer, answers to the twenty questions, and the path ahead

Written for the Leaper owner, 4 September 2026. Built from the repository's ledger (TRAINING.md, AGENTS.md, RL_SPEC.md, the run logs), the environment and trainer source code, and the question dossier Leaper: target memory and the LSTM runs.

THE WHOLE DOCUMENT IN FIVE SENTENCES

Leaper already walks and dodges rocks very well, and that skill lives in a small brain that must not be thrown away. The memory brain (LSTM) lost because it replaced that whole brain with a much bigger one that had to relearn walking from scratch while also learning to remember. The fix is to keep the small walking brain and hand it memory as extra written notes from the game (a 'cleared map' of where she has already looked, and a 'target note' that later carries the target's direction of travel). When a learned memory is wanted for the moving target, the owner's two-branch brain (the plain walker plus a small LSTM that sees only the target stream) is the right way to add it. The next experiment is one cheap, controlled run: PPO_29 plus the cleared map, three seeds, with a written pass mark of 78%.

How to read this document

- **Part 1** explains the words. Skim it, come back to it whenever a term is unclear.
- **Part 2** tells the story of what has been tried and what each run taught, using the same numbers as the ledger.
- **Part 3** explains what the memory brain really is, why it was expected to help, and why it did not.
- **Part 4** introduces the two ideas this document rests on: memory as notes written by the game, and the owner's two-branch brain (a plain walker plus a small memory branch).
- **Part 5** answers the twenty questions one by one. Each has a short answer first, then the reasoning, then what to do.
- **Part 6** lays out the path ahead as concrete experiments with pass marks.
- **Part 7** is a one-page cheat sheet you can keep next to the keyboard.

Part 1. The words, in plain language

Reinforcement learning has its own vocabulary, and most of the confusion in this project comes from words that sound grander than what they mean. Here is the whole vocabulary this document uses. Everything else is explained where it appears.

Word	What it means for Leaper
Brain / policy	The thing that decides what to do. It takes in a list of numbers (what Leaper senses) and outputs two numbers (how hard to push forward, how much to turn). It is a table of arithmetic with adjustable knobs.
Knobs / parameters / weights	The adjustable numbers inside the brain. Training means nudging these knobs. PPO_29's brain has about 12,000 knobs. The memory brains had 623,000 and 1,676,000.
Observation / senses	The 26 numbers Leaper gets every step: is the target in view, how long since she saw it, which way and how far it was, her heading, whether she just bumped something, her last move, and 16 distance readings from rays fanned across her front 270 degrees.
Action	The two output numbers: throttle (forward speed) and turn.
Step	One decision. Leaper moves at most 0.375 units and turns at most 9 degrees per step.
Episode	One attempt in one arena, from a random start until she reaches the target, fails, or 1,000 steps run out (a 'timeout').
Reward	Points handed out every step by the game rules: +25 for reaching the target, +0.5 for spotting it the first time, small amounts for progress, small costs for time, bumping and standing still, and a one-time -10 for getting wedged or freezing.
PPO	The coaching method. Leaper tries things, the coach looks at which tries scored well, and nudges the knobs toward those. Repeat thousands of times.
Training steps (200k, 500k)	How many decisions Leaper practised during training. Not a quality measure, just the practice budget.
Checkpoint	A saved copy of the brain every 10,000 practice steps.
Seed	The random-number starting point. Two trainings with different seeds are like two students taught the same syllabus: they end up similar but not identical.
Deterministic vs stochastic	Deterministic means Leaper takes her single best guess every step; that is how she plays in the game. Stochastic means dice are added to each move; that is how she explores during practice. Only deterministic results count as the score of record.
The 100-maze exam	The same 100 fixed arenas (seeds 10,000 to 10,099), played deterministically. Every run sits the identical exam, so scores are comparable.
Plain brain (MLP)	A brain with no memory. Each step it looks at the current 26 numbers and answers. Yesterday does not exist for it.
Memory brain (LSTM)	A brain with a built-in notepad that it rewrites every step and reads the next step. The notepad's contents are learned and unreadable by humans.
Warm start / warm transfer	Starting a new training from the old brain's knobs instead of from random knobs, so previous skill carries over.

Word	What it means for Leaper
Actor and critic	Two halves that PPO trains side by side. The actor is the brain that acts. The critic is a scorekeeper that guesses 'how well is this going?' and is only used during training, never in the game.
State augmentation	Adding extra hand-computed numbers to the observation, written by the game engine, instead of asking the brain to work them out itself. The current 'last seen target' note is already an example.
Belief	The brain's working estimate of something it cannot currently see, for example 'the target is probably behind that rock and drifting left'.

FOUR PAIRS THAT ARE EASY TO MIX UP

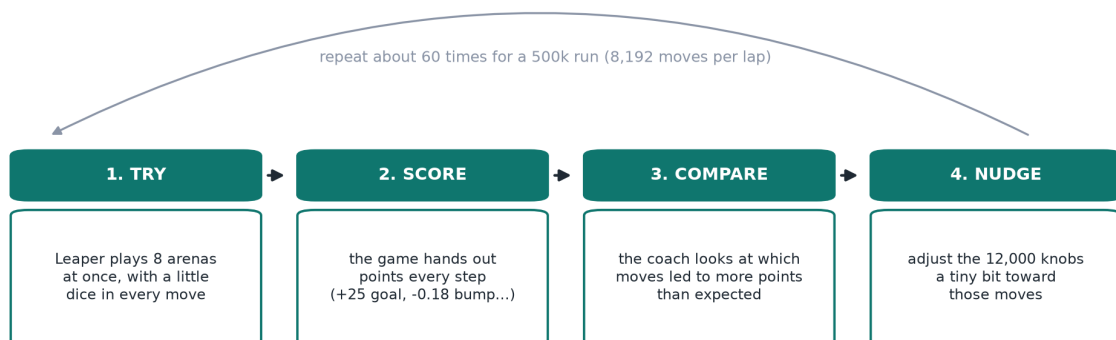
Exam versus spot check. The exam is 100 fixed mazes and is the score of record. A spot check is 25 mazes every 10,000 practice steps and is only for watching the trend.

Deterministic versus stochastic. Deterministic is how she plays; stochastic is how she practises. Only deterministic numbers are compared between runs.

Head versus branch. A head is an output (action head, value head). A branch is a separate pipe of processing inside the brain. The owner's two-branch idea has two branches and the usual two heads.

Memory brain versus a note. A memory brain learns its own private way to remember. A note is a plain number written by the game engine (for example 'target last seen 30 steps ago'). Both are memory; only one is readable.

How training works, in one loop



Every training run is this loop repeated. 'PPO' is just the name of the rule used in steps 3 and 4.

Two consequences of this loop explain a lot of the history. First, the coach can only prefer moves that actually happened, so the dice in step 1 matter: if the dice never push the throttle above zero, a freeze cannot be unlearned (that was PPO_28). Second, every knob is nudged on every lap, so a bigger brain is not just slower to train, it is more likely to wobble: more knobs, same evidence.

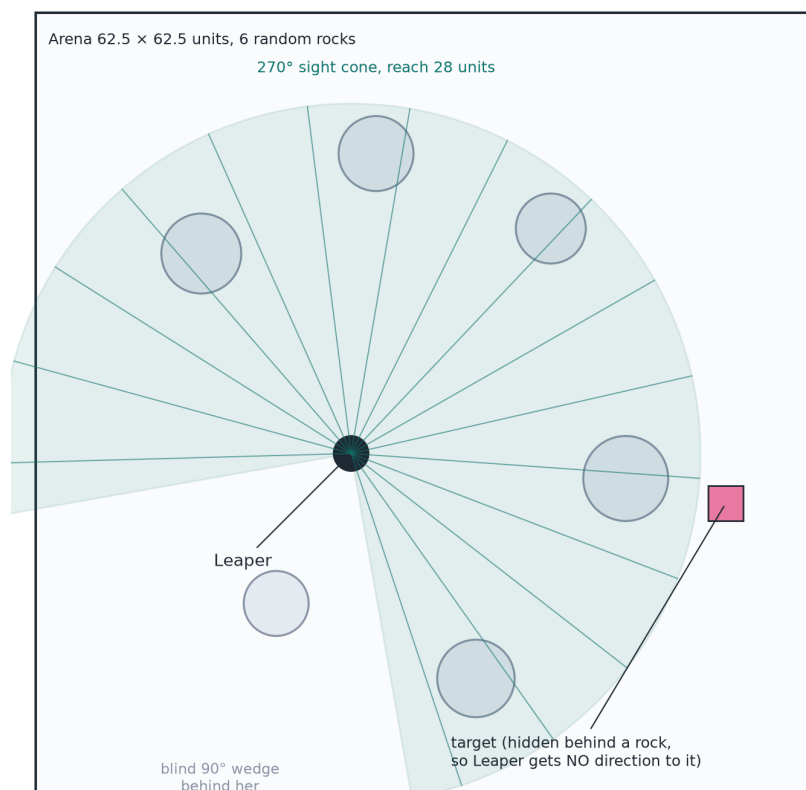
Part 2. The story so far

Three skills that look like one

From the outside, 'Leaper reached the target' is one event. From the inside it is three separate skills, and the whole history of the project is easier to follow once they are pulled apart:

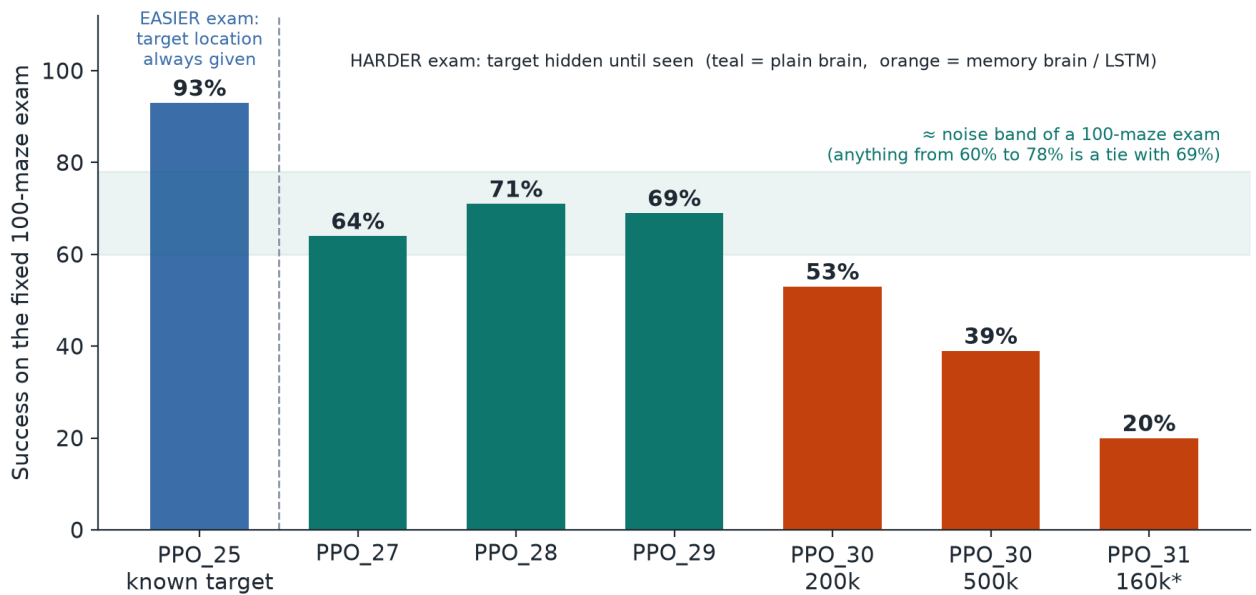
- 1 Walking and dodging.** Move decisively, steer around rocks, get out of contact, never freeze. This is solved, and solved well.
- 2 Searching.** When the target is not in view, cover the arena efficiently until it comes into the 270-degree cone. This is the current blocker.
- 3 Tracking and pursuit.** Once seen, keep going toward the target even when a rock hides it for a while. Later, when the target moves, predict where it is going. This is the next goal.

Up to PPO_25 the game handed Leaper the exact direction and distance to the target on every step, so skills 2 and 3 were never tested. From PPO_27 onward that information is hidden until she genuinely has line of sight. That is why the two families of scores must never be compared directly.



The seeker world. The arena is 62.5 units wide with six random rocks. Leaper sees through a 270-degree cone that reaches 28 units, using 16 thin distance rays. The target is only reported when it is inside the cone, within reach, and not behind a rock.

What the numbers say



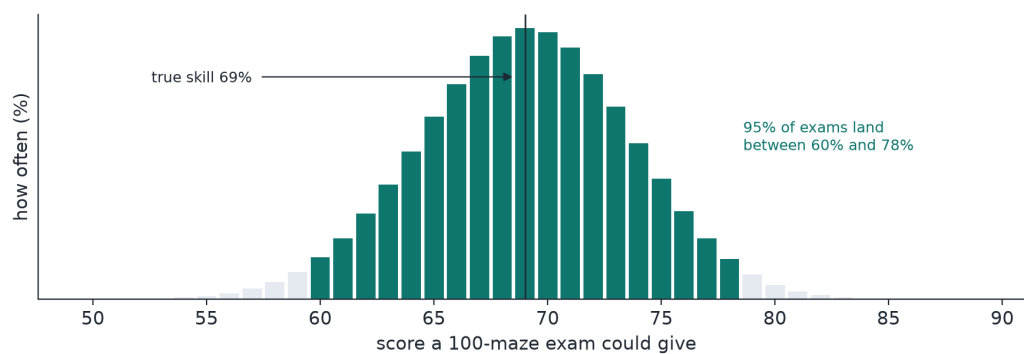
*PPO_31: 25-maze checkpoint, not a final exam

Deterministic success on the fixed 100-maze exam. Blue is the easy exam (target location always given). Teal and orange are the hard exam (target hidden until seen). Teal runs use the plain brain, orange runs use the memory brain.

A NUMBER YOU NEED FOR EVERYTHING BELOW: THE NOISE BAND

A score from 100 episodes is like 100 coin flips with a weighted coin. If the true skill is 69%, the exam will land anywhere between about 60% and 78% just by luck. So PPO_27 (64%), PPO_28 (71%) and PPO_29 (69%) are **ties**. PPO_30 at 39% and PPO_31 at about 20% are genuinely worse. Twenty-five-episode spot checks during training are four times noisier again: one episode is four points.

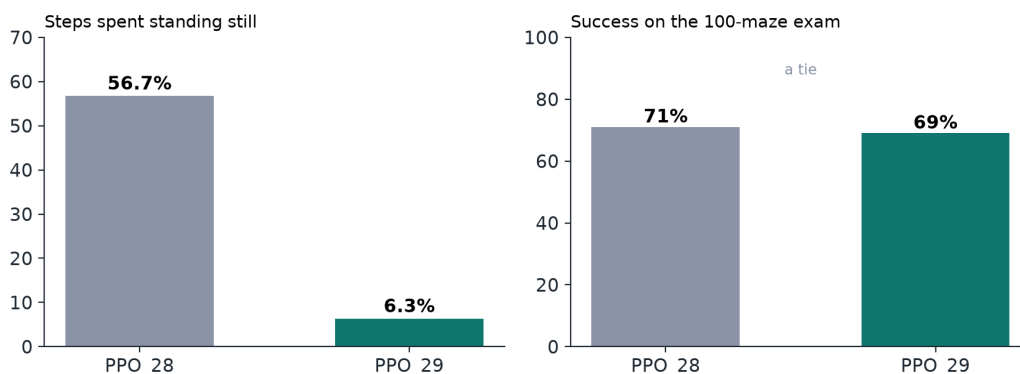
Rule of thumb: to claim a real improvement on the 100-maze exam you want at least eight to ten points, or the same direction across three training seeds.



If Leaper's true skill were exactly 69%, this is how often each exam score would come up. The teal bars cover 95% of exams.

What each milestone taught

Run	What changed	Exam	Lesson
PPO_25	Known target, 16 rays, forward-only, idle penalty	93%	Walking and dodging are solved. Confirmed on 1,000 fresh arenas (91.3%).
PPO_27	Target hidden until seen; smaller arena; warm-started from PPO_25	64%	Search is learnable. But rewarding every re-sighting (+0.5) let her circle at the edge of sight to farm points.
PPO_28	Half speed and turn, double episode length, +0.5 for first sight only	71%	The circling exploit is gone. But she stood still 57% of the time, almost always before she had ever seen the target.
PPO_29	Throttle output re-centred so 'neutral' means half speed; 60-step freeze rule; small reward for one scan	69%	Freeze cured (57% down to 6% standing still). Success unchanged. So freezing was a symptom. The real ceiling is search: about 30% of episodes never find the target.
PPO_30	Memory brain (one LSTM layer of 256), trained from scratch	53% then 39%	Worse than the plain brain and unstable: spot checks wobbled 44 to 68% and the final save landed on a bad moment.
PPO_31	Two LSTM layers, five times larger exploration reward, fewer update passes	about 20%	Three changes at once, so nothing can be attributed. Episodes ballooned to 640 steps: she learned to wander for exploration points. Process ended at 164k for an unrecorded reason.



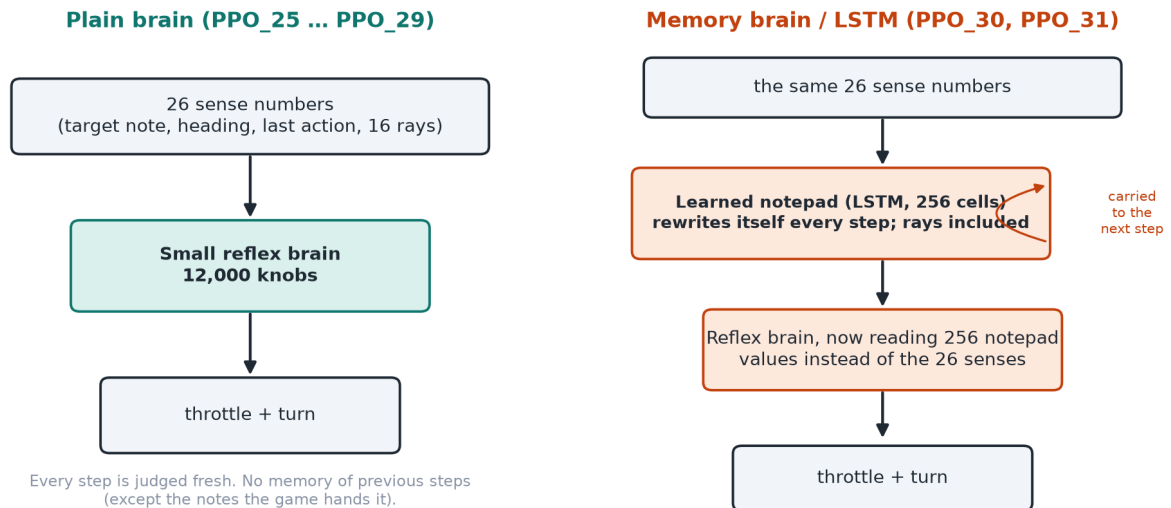
PPO_29 in one picture: the freeze fix worked completely, and it changed the success rate by nothing. That is the evidence that the remaining problem is search, not movement.

Why search is the blocker, in numbers

It helps to see how generous the seeker world already is. From the middle of the arena, one full turn with a 28-unit reach exposes roughly six tenths of the floor. An episode allows 1,000 steps at 0.375 units each, which is 375 units of travel, about six arena widths. Leaper has plenty of eyes and plenty of time. What she lacks is a plan: with no record of where she has already looked, a plain brain wanders back through cleared ground and can spend a whole episode never facing the one corner where the target sits. That is exactly the gap that memory is supposed to fill, and it is why the memory brain's failure was such a surprise. Part 3 explains it.

Part 3. What the memory brain is, and why it lost

Two kinds of brain

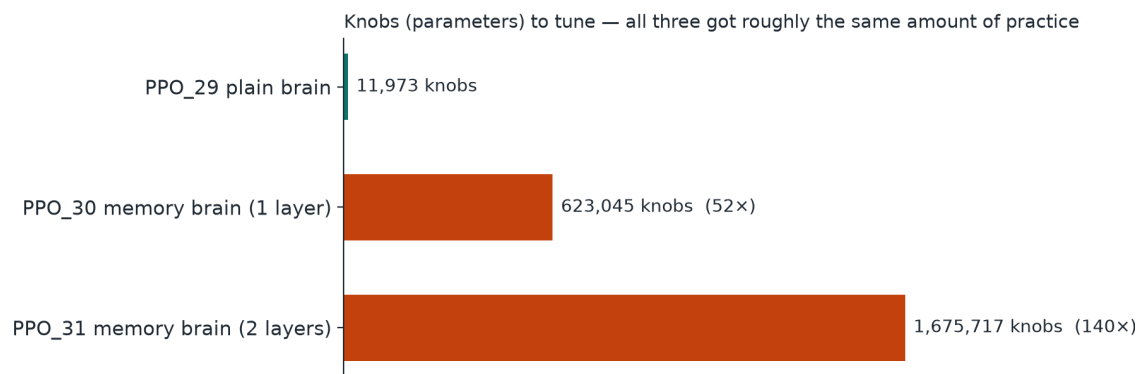


Left: the plain brain used from PPO_25 to PPO_29. Right: the memory brain used in PPO_30 and PPO_31. Notice that in the memory brain the notepad sits in front of everything, including the 16 obstacle rays.

A **plain brain** is a reflex. Each step it receives the 26 numbers and produces throttle and turn. It has no idea what happened one step ago except for the few 'notes' the game hands it (last move, last bump, last-seen target).

A **memory brain** adds a notepad of 256 numbers. Every step it reads the 26 senses and the notepad, writes a new notepad, and then decides. In principle it can keep anything it likes on that notepad: where it has been, what it saw, how long ago. In practice, three things about the way it was installed here mattered enormously:

- **The notepad was placed in front of everything.** Even the obstacle rays go through it. The old reflex 'ray on the left is short, so turn right' now has to pass through a learned, constantly rewritten memory before it can act. A shaky memory can distort a reflex that already worked perfectly.
- **The old brain could not be carried over.** The plain brain's first layer reads 26 numbers; the memory brain's reflex part reads 256 notepad numbers. The shapes simply do not match, so PPO_30 and PPO_31 had to learn to walk and dodge from scratch while also learning to remember. Everything PPO_25 through PPO_29 had earned was left on the table.
- **It was 52 to 140 times bigger.** Same amount of practice, vastly more knobs to settle. Big brains need more data and are more prone to wobbling, which is exactly what the checkpoints showed.

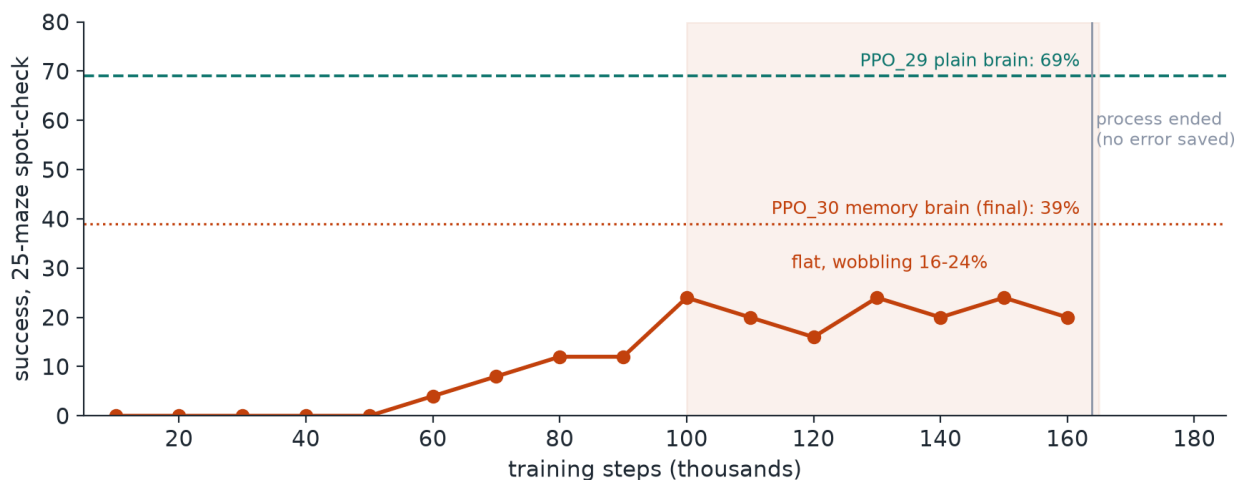


The plain brain versus the two memory brains, by number of adjustable knobs.

Why it seemed to 'affect the eyes and the movement'

It did, and it was not a bug. Because the notepad sits between the senses and the reflexes, every action depends on the accumulated notepad as well as on what the rays currently show. The critic (the training-time scorekeeper) has its own separate notepad, so the learning signal itself was also being filtered through a memory. Nothing of the old walking brain sat intact underneath. This also explains every 'only some tensors match' message: there is no layer-by-layer correspondence to load into.

PPO_31 specifically



PPO_31's 25-maze spot checks. Learning started late, peaked at 24%, and wobbled. The process ended after the 163,840-step rollout with a clean log and no error message.

PPO_31 changed three things at once (two notepad layers instead of one, a five-times larger reward for entering new cells, and half the update passes per batch). Any of the three could explain the result, so PPO_31 cannot tell us whether two layers are worse than one. What it does show clearly is a reward loophole: episodes grew from about 180 steps to over 640, and the diagnostics say she spent her time collecting 'new cell' points rather than committing to finding the target. Bigger exploration bonuses reward wandering.

On the stopping: the ledger calls it 'almost certainly out of memory'. The honest statement is that there is no evidence either way. No exception was logged, no non-finite values appear in TensorBoard, and the run status still says 'training'. That pattern (clean log, no traceback) is consistent with the operating system killing the process, and a two-layer LSTM

plus TensorBoard plus a browser is a plausible memory hog, but it is a guess. It is definitely not a 'neural network explosion'. Phase 0 below closes this properly.

The most likely reasons the memory brain lost, ranked

- 1 Relearning the body from scratch.** The strongest predictor in this whole project: every run that kept the walking weights did fine, every run that started from nothing on the hard task struggled. PPO_30 and PPO_31 threw away the walker.
- 2 Too many knobs for the data.** 52 to 140 times more knobs, same practice budget. Recurrent PPO is also known to be finicky; the oscillating checkpoints in PPO_19 to 21 and PPO_30 are the classic signature.
- 3 Search coverage was never shown to the brain.** The reward system knows which cells were visited (it pays for new ones), but the observation never shows that map. The notepad had to reconstruct 'where have I been' from hundreds of steps of headings and rays. That is a very hard job to learn by trial and error, and it is not what the reward directly pays for.
- 4 The rays went through the notepad,** so the obstacle reflex could be corrupted by memory noise.
- 5 The target note already existed.** The game already remembers the last-seen target for 120 steps. The notepad's only real added value was long-range search memory, the hardest thing for it to learn.
- 6 Reward competition (PPO_31 only).** Big exploration bonuses made wandering profitable.

THE TAKEAWAY

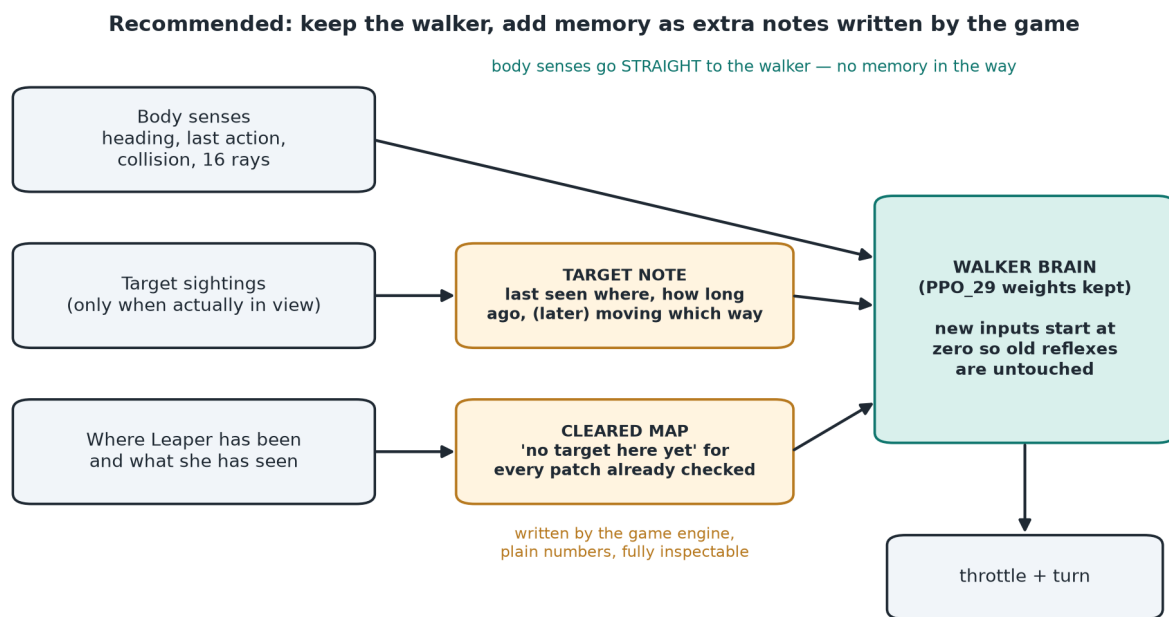
Memory as an idea is not disproven. What was disproven, twice, is one specific way of adding it: replacing the whole brain with a big learned notepad and training from zero. The rest of this document is about adding memory without doing that.

Part 4. The big idea: memory does not have to live inside the brain

There are two ways to give Leaper a memory.

- **Inside the brain (LSTM):** a learned notepad in the brain's own handwriting. Flexible in theory. Expensive, unstable, and unreadable in practice, as seen.
- **Outside the brain (state augmentation):** the game engine keeps the notes in plain numbers and hands them to the brain as extra senses. The brain stays small and plain. This is the same trick the project already uses for 'last-seen target' and 'how long ago'. It just has not been used for search coverage yet.

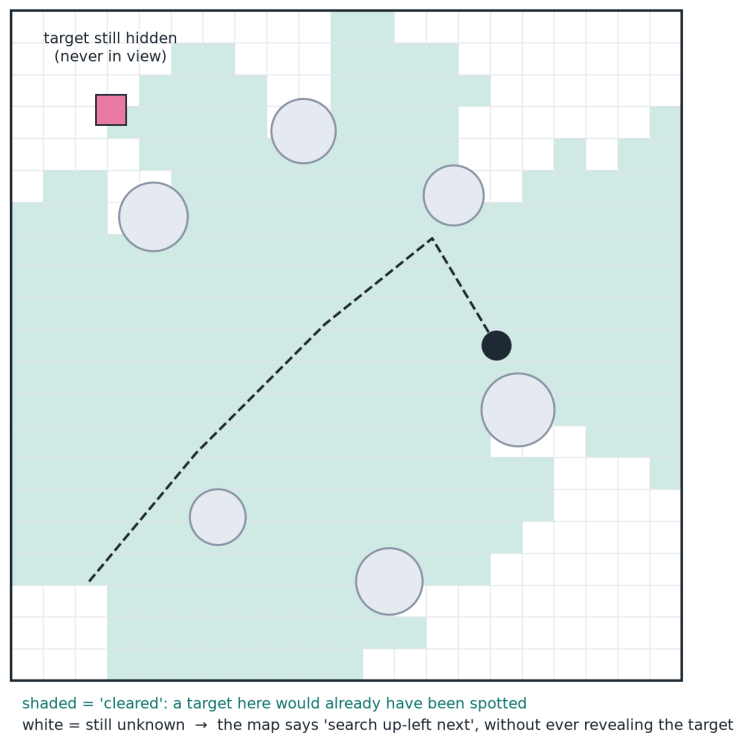
Outside-the-brain memory has a property that matters more than anything else here: **the walker survives**. Adding new senses to a plain brain means adding a few new input columns. Set those new columns to zero at the start and PPO_29's weights behave exactly as before on day one. Training then only has to learn 'and also steer toward the unexplored side'. This is exactly how PPO_25 became PPO_27 (two repurposed inputs, zeroed, optimizer reset) and it worked.



The recommended shape. The walker keeps its weights. Two notes are written by the game engine: the target note (already exists; later gains direction of travel) and the cleared map (new). Body senses go straight to the walker with no memory in the way.

The cleared map, honestly built

The map must never leak the answer. The rule that makes it honest is simple: **a patch of floor is marked 'cleared' only when the target would have been reported had it been there**, that is, the patch was inside the cone, within 28 units, and not behind a rock. Unseen patches stay blank. The target's position is never touched. The map can only ever say 'not here yet', never 'there'.



A cleared map part-way through an episode. Shaded cells have been checked; white cells have not. The target sits in an unchecked corner. The map does not know that, but it does know that the up-left region is the only place left to look.

How the map reaches the brain matters too. Handing over the whole grid (441 cells) is possible but wasteful. A compact, Leaper-centred summary works better with a small brain and matches the way she already sees: **16 'unexplored' rays** in the same directions as the 16 vision rays, each reporting how far she would have to travel in that direction to reach unchecked ground (0 = right here, 1 = nothing unchecked within 28 units). Sixteen numbers, same layout as her eyes, rotation-consistent, and the observation grows from 26 to 42.

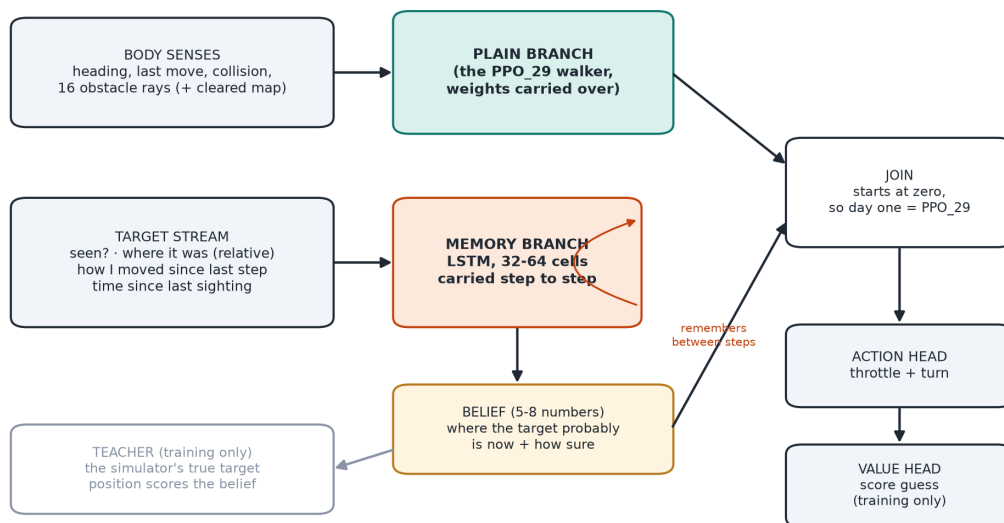
WHY THIS IS THE RIGHT FIRST TEST

It attacks the diagnosed blocker (search) directly. It keeps the champion walker intact. It costs about ten minutes per training run on the plain brain instead of 90 minutes for a memory brain. It is inspectable: you can draw the map in the replay viewer and see whether she is heading for the white space. And if it fails, that failure is informative: it would mean the missing piece is not coverage memory at all.

The second idea: a two-branch brain

The owner's proposal: keep the ordinary brain for the ordinary senses, and add a second, separate LSTM that sees only the memory-related information. In machine-learning language this is a **multi-branch** network (people sometimes loosely say 'multi-head'). It is a good idea, and it is the healthiest way to put an LSTM into Leaper. This section explains what it is, how it would work, and where it fits.

The two-branch brain: a plain walker plus a small target-only memory



The rays never touch the memory. The memory never touches the rays.

The two-branch brain. Body senses go through the plain branch (the PPO_29 walker). Only the target stream goes through the small memory branch. The two meet at a join that starts at zero, then feed the usual action and value heads.

What 'multi-head' or 'multi-branch' means

A normal network is one pipe: everything goes in one end, actions come out the other. A multi-branch network has two or more pipes running side by side. Each pipe is fed a different slice of the senses and can be built differently (one plain, one with memory). The pipes meet at a join before the final decision. The word 'head' is best kept for the outputs, which PPO already has two of (action and value). So the design reads: two branches in, one join, two heads out.

How it works, step by step

- 1 Body senses** (heading, last move, collision, the 16 rays, and the cleared map if present) go into the **plain branch**. This is the PPO_29 walker with its weights carried over unchanged.
- 2 The target stream** (seen or not, where it was relative to Leaper, how Leaper herself moved since the last step, time since the last sighting) goes into the **memory branch**: one small LSTM of 32 to 64 cells whose contents are carried from step to step and wiped at the start of each episode.
- 3** The memory branch outputs a **belief**: five to eight numbers meaning 'the target is probably over there, moving that way, and I am this sure'.
- 4** The **join** puts the walker's features and the belief side by side. The connections coming from the belief start at zero, so on day one Leaper behaves exactly like PPO_29 and only gradually learns to use the belief.
- 5** The usual two heads sit after the join: the **action head** (throttle, turn) and the **value head** (score guess, used in training only).

What happens inside an LSTM cell, in plain words

Each cell keeps one number between steps. Every step, three tiny learned gates decide how much of the old number to **forget**, how much of the new input to **write**, and how much of the stored number to **reveal** to the rest of the brain. That is the whole trick. 'Long short-term memory' just means the gates can let a number survive for hundreds of steps if the network learns it is worth keeping. With 32 or 64 such cells you have a small notepad that can hold something like 'last seen at bearing 40 degrees, 30 steps ago, drifting left'.

Why this version can work where PPO_30 and PPO_31 failed

- **The rays never enter the memory.** The obstacle reflex cannot be distorted by a confused notepad.
- **The walker's weights carry over.** Nothing has to relearn walking and dodging.
- **It is about fifty times smaller.** A 64-cell LSTM on a handful of inputs is roughly 20,000 knobs, against 623,000 for PPO_30.
- **It can be taught, not just rewarded.** See the teacher trick below.

The teacher trick

The simulator knows the true target position every step. So the memory branch can be scored directly on how close its belief is, every single step, alongside the usual PPO learning. This is supervised learning, and it is far steadier than waiting for the +25 at the end of an episode to tell the memory whether it remembered well. The walker still learns by PPO exactly as before. The teacher is used only in training; in the game the branch runs on its own.

Where it helps, and where it does not

The branch only knows what it has seen. Before the first sighting it is empty, so for the static hidden target it mostly duplicates the 120-step note the game already keeps. Its value appears with a **moving** target and with occlusions, where 'which way was it going' matters and a straight-line guess may not be enough. Search coverage still needs Leaper's position history, and that stays in the explicit cleared map feeding the plain branch.

Note that the diagram does not change if the orange box is replaced by plain arithmetic (store last position, estimate velocity, fade confidence). Same wiring, zero learned knobs. That arithmetic version is Phase 2; the LSTM version is Phase 3. Build the wiring once, swap the middle box, and you have the cleanest possible comparison.

What it costs

Stable-Baselines3 has nothing off the shelf for two branches with different inputs, so it needs a custom policy class (a few hundred lines) with its own tests. Training is slower than the plain brain but far faster than PPO_30: a few seconds per 1,000 steps, not fifteen. The browser export must carry the LSTM's numbers between frames, and the fixture self-test must include them.

WHERE IT FITS

Phase 3 of the plan in Part 6. Build it after the cleared-map seeker (Phase 1) and the arithmetic target note (Phase 2) exist, then race it against the arithmetic note on the same seeds and budget. Building it sooner is a fair call if the owner wants to; the one thing not to skip is the arithmetic baseline, because without it nobody can tell whether the LSTM branch adds anything.

Part 5. The twenty questions, answered

Each answer starts with the short version, then the reasoning, then a 'what to do' box where there is something concrete to do. Answers refer back to Parts 3 and 4 rather than repeating them.

Question 1

Can this project preserve its learned local-navigation policy while adding a separate memory mechanism for target search and tracking? What is impossible with the off-the-shelf memory policy, and what is possible with a custom modular or hierarchical policy?

SHORT ANSWER

Yes, it can be preserved. With the off-the-shelf memory policy (sb3-contrib's `MlpLstmPolicy`) it is impossible, because that policy puts the notepad in front of the whole brain and changes the shape of every layer that follows. With a custom policy it is possible, and with outside-the-brain memory it is not just possible but trivial: you widen the observation and warm-start from PPO_29.

The off-the-shelf memory policy is a single block: senses go into an LSTM, the LSTM's 256 outputs go into the reflex layers. There is no option in that block for 'let these 26 numbers bypass the notepad'. So the old first layer (26 inputs) can never be loaded into the new first layer (256 inputs). That is the whole reason PPO_30 and PPO_31 had to start from nothing.

A custom policy in Stable-Baselines3 (a custom features extractor or a custom actor-critic class) can route the body senses straight to the reflex layers and route only the target stream through a small notepad, then join the two. That preserves the walker by architecture. It is a real piece of engineering, a few hundred lines, and it needs its own tests.

Outside-the-brain memory needs none of that. The game writes the notes, the plain brain reads 42 numbers instead of 26, and the transfer is the same zero-the-new-columns trick that already worked from PPO_25 to PPO_27. This is the recommended route for the static target and for the first moving-target stages.

The owner's two-branch brain (Part 4) is the custom-policy route done right: the walker branch keeps PPO_29, the memory branch sees only the target stream, and the two meet at a join that starts at zero.

WHAT TO DO

Treat 'preserve the walker' as a hard requirement for every future run. Any design that cannot load PPO_29's weights on day one should need a written justification before it is trained.

Question 2

What architecture would best isolate responsibilities? Should obstacle rays and immediate locomotion remain feed-forward while only target observations, target motion, and search history enter a memory branch?

SHORT ANSWER

Yes. Rays, heading, last action and collision go straight to the walker with no memory in the way. Only the target stream and the position history feed memory. And for now, make that memory explicit (written by the game) rather than learned.

The reasoning is in Part 3: the obstacle reflex is a here-and-now problem, so memory can only hurt it. Target motion is a genuinely temporal problem (you need at least two sightings to know a direction of travel), and search coverage is a position-history problem. Those two deserve memory. Neither needs the rays.

The cleanest layout is three pieces with clear jobs: a **tracker** that answers 'where is the target probably now', a **cleared map** that answers 'where have I not looked', and the **walker** that turns those plus the rays into throttle and turn. The walker is the only piece that needs reinforcement learning. The other two can be arithmetic.

GOES INTO THE TARGET MEMORY

- target seen? (yes/no)
- where it was, relative to Leaper
- how Leaper herself moved since (step + turn)
- how long since the last sighting

STAYS OUT (goes straight to the walker)

- the 16 obstacle rays
 - collision flag
 - last throttle / turn
- (unless a controlled test proves they help)

WHY: target motion can be worked out from sightings + Leaper's own movement alone. Rays describe rocks, not the target; feeding them into memory lets a confused memory distort an obstacle reflex that already works. Search coverage needs POSITION history, which is far better kept as an explicit map than squeezed through a learned notepad.

What belongs in the target memory and what stays out.

Question 3

Is a 'two-head' design the right concept, or should this be a tracker, planner, and controller hierarchy? Where should actor, critic, memory state, and navigation features live?

SHORT ANSWER

'Two heads' is the wrong picture. In PPO the two heads already exist and mean 'action' and 'score estimate'. What you want is a tracker plus a controller, with a planner added later only if needed. Memory state lives in the tracker and the map, navigation features go straight to the controller, and the critic is free to see more than the actor does.

Concretely: **the tracker** holds the target belief (last seen where, moving which way, how long ago, how confident). Start with an explicit filter (arithmetic), replace with a small learned notepad only if the filter fails. **The cleared map** holds search history. **The controller** is the PPO_29 walker reading rays + heading + last action + collision + tracker output + map summary. It has the usual two heads: the actor outputs throttle and turn, the critic outputs a score estimate.

A separate **planner** (something that picks a waypoint every 20 or 50 steps and hands it to the walker) is the fully hierarchical version. It is the most protective of the walker and the

most work. Keep it in reserve for the moving-target stage, and only if the flat version stalls. One detail that costs nothing and helps a lot: **the critic may cheat**. It is never deployed, so during training it can be given the true target position or the full map. This is called an asymmetric critic and it steadies learning under hidden information. The actor still only sees honest senses, so the game stays fair.

WHAT TO DO

Name the pieces tracker, map, walker in the code and the ledger. Stop using 'two heads' for the memory idea; it invites confusion with PPO's action and value heads.

Question 4

Which pretrained weights can safely be reused? PPO_25 is the known-target champion and PPO_29 the strongest hidden-target brain. Which is the better donor, which layers should be frozen, which retrained?

SHORT ANSWER

PPO_29 is the donor for everything in the seeker line. PPO_25 only becomes relevant if a separate waypoint-following walker is built later. Freeze nothing permanently; protect the walker with zeroed new columns, a fresh optimizer, and a modest learning rate, and watch the body-health metrics at every checkpoint.

PPO_29 already knows the seeker world: the hidden-target observation, the re-centred throttle, the freeze rule, and what to do before the target has ever been seen. PPO_25 knows none of that; it was trained with the answer always given and has no 'not found' mode. Warm-starting the seeker from PPO_25 would repeat the PPO_27 step, which worked but cost a whole run to relearn search basics.

Reuse: all of PPO_29's actor and critic weights, shape for shape, with new input columns added as zeros. Retrain: everything, gently. The critic in particular must relearn, because the value of a state changes once the map exists.

Freezing is insurance, not a first choice. If a run shows the walker degrading (collision rate or standing-still rate creeping up at the 25-maze checkpoints), freeze the old weights for the first 50k steps so only the new columns and the last layer move, then unfreeze.

WHAT TO DO

Donor: `rl_artifacts/ppo_29_normalized_throttle_250k/leaper_ppo.zip`. New input columns zero-initialised, optimizer reset, learning rate $1.5e-4$ to $3e-4$. Add 'collision %', 'stopped %' and 'success after first sight' to the checkpoint printout as a body-health check.

Question 5

How should the observation interface be designed so transferred navigation weights keep their meaning? How should target-visible, target age, predicted direction and distance, velocity, heading, previous action, collision, and the 16 rays be routed?

SHORT ANSWER

Keep the first 26 numbers exactly as they are, in the same order and with the same meaning. Append every new number after index 25. Never reorder, never change a scale. Then the old weights keep their meaning automatically.

The plain brain's first layer is a table with one column per input number. Weight columns are tied to positions, not to names. If index 10 stops meaning 'leftmost ray', the walker is blind. So the contract is: indices 0-25 are frozen in meaning forever in the seeker line, and growth happens at the end.

Proposed layout for the seeker with a cleared map (42 numbers): indices 0-25 unchanged; 26-41 the sixteen 'distance to unchecked ground' rays, same directions as the vision rays, normalized 0-1.

Proposed further growth for a moving target (49 numbers): 42-43 estimated target velocity (x, z, normalized by Leaper's speed); 44 confidence (1 just after a sighting, decaying toward 0); 45 observed-or-predicted flag; 46-47 predicted direction to the target now (unit vector); 48 predicted distance. Indices 2-4 keep showing the raw last-seen direction and distance, so the walker always has both the memory and the prediction and can learn which to trust.

All values stay within -1 to 1, matching the existing scaling, so nothing needs re-normalizing.

WHAT TO DO

Write the index table into RL_SPEC.md before touching code, and add a test that fails if any of indices 0-25 changes meaning (the existing brain_fixtures.json self-test is the right pattern).

Question 6

Should the next baseline use explicit state augmentation rather than an LSTM? Would last-seen position, estimated velocity, confidence, and prediction through occlusion be sufficient for the first moving-target stages?

SHORT ANSWER

Yes on both counts. Explicit augmentation is the next baseline. For a target that moves slowly and smoothly, last-seen position, estimated velocity, age, confidence, and a straight-line prediction are enough for the first stages. That is how real trackers (radar, video) handle short occlusions.

A straight-line prediction is honest and cheap: when the target is visible, update its position and velocity from two sightings; when it is hidden, move the estimate forward at the last velocity and shrink confidence every step. After a long enough occlusion (for example 120 steps, matching today's memory horizon) confidence hits zero and the note flips back to 'not found', so the search behaviour takes over again.

Where this stops being enough: a target that changes direction while hidden, or that actively evades. That is when a learned tracker or a planner earns its place. Do not build for that case until the simple filter has been shown to fail on it.

WHAT TO DO

For the static target: cleared map only. For the first moving target: the target note gains velocity, confidence, prediction, as in Question 5. Measure tracking error separately from success (Question 18).

Question 7

Does choosing state augmentation now create a future migration problem? If an LSTM becomes necessary later, can it consume the augmented belief state without destroying the existing navigation behaviour?

SHORT ANSWER

No migration problem. Augmentation and a small memory branch are complementary. A later notepad can read the same notes and add its own output as more appended inputs to the same walker. The only dangerous move is the one already made twice: replacing the whole brain.

Think of the observation as a growing list of senses. Adding a learned notepad later means appending its 32 or 64 outputs to that list, zeroed at first, exactly like appending the map. The walker never notices until training teaches it to use the new columns.

The notes are also the perfect training signal for a learned tracker: because the simulator knows the true target position, a small notepad can be taught to predict it directly (supervised learning, which is far more stable than learning it through rewards). The explicit filter you build now becomes the baseline the learned tracker has to beat.

WHAT TO DO

Keep a written rule in AGENTS.md: 'new memory is appended to the observation; the walker is never replaced'.

Question 8

What should the memory actually receive? Only target measurements and robot ego-motion, or also obstacle rays and collision/action history? What is necessary for target-motion inference versus map/search memory?

SHORT ANSWER

Target memory needs only: seen-or-not, the target's position relative to Leaper when seen, Leaper's own step and turn since the last step, and time since the last sighting. Search memory needs Leaper's position history plus the viewing geometry. Neither needs the rays.

Target motion inference is geometry: two sightings plus knowledge of how Leaper herself moved between them gives the target's velocity. Leaper's own movement is essential (otherwise the tracker cannot tell 'target moved left' from 'I turned right'), and it is already available as last throttle and turn, or better, as an exact position delta from the engine.

Search memory is the record of which patches have been checked. That is a function of Leaper's positions and headings over time plus what blocked her view. The engine knows all of it exactly, which is why an explicit map beats a learned notepad here.

The rays describe rocks. Rocks matter for occlusion (why did the target vanish?), but the tracker does not need to know why, only that it did. If someone wants to test rays in the tracker, that is a controlled comparison, not a default.

WHAT TO DO

Give the tracker exactly the inputs in the figure under Question 2. Log Leaper's true position delta from the engine rather than relying on the previous action, because a blocked move is not a move.

Question 9

What memory size and type are proportionate? Would a one-layer 32 or 64-unit GRU or LSTM be more appropriate than separate 256-unit actor and critic LSTMs? Should actor and critic share a tracker state?

SHORT ANSWER

If a learned memory is added, a single-layer GRU or LSTM of 32 to 64 units, used only for the target stream (the two-branch brain in Part 4), is proportionate. The actor and critic should share one tracker and keep separate heads. Two separate 256-unit notepads over the full observation was the wrong scale by roughly an order of magnitude for this task.

Size should match the job. The target note is five to eight numbers; a 64-unit memory to produce it is already generous. A GRU is slightly simpler and smaller than an LSTM and trains a little more easily at this scale; either is fine.

Sharing the tracker halves the memory and, more importantly, means actor and critic agree about where the target probably is. sb3-contrib supports this directly (`shared_lstm=True` with `enable_critic_lstm=False`), though that still routes the whole observation through the notepad, so a custom policy is needed to get the target-only routing.

For the immediate work none of this applies: the explicit filter has zero learned knobs.

WHAT TO DO

Do not build a learned tracker until the explicit one fails a measurable test (Question 19).

Question 10

How should memory be trained and reset? What sequence length, burn-in, truncation, episode-start handling, and curriculum suit occlusions lasting tens to hundreds of steps?

SHORT ANSWER

For explicit memory: nothing to train, reset the notes at every episode start. For a learned tracker, train it with supervised targets from the simulator (it knows the true target position), reset its state at episode start, use sequences of 128 to 256 steps, and grow occlusion length gradually.

The current recurrent trainer already resets the notepad at each episode start using the episode-start mask, and it trains on sequences cut from the 1,024-step rollouts. That part was fine. There is no burn-in in sb3-contrib's implementation; the stored hidden state from the rollout is used as the sequence's starting point, which is acceptable for sequences this long.

The reason 'learning to remember through rewards' is hard is that the reward arrives once, far in the future. Supervised training of the tracker (predict the target's true position, be scored on the error every step) gives a signal every step and is stable. The walker is then trained by PPO with the tracker's output as a sense, as before.

Curriculum for occlusions: start with the target visible; then hidden for up to 30 steps; then up to 120; then longer. Advance only when the deterministic exam is stable at the current level.

WHAT TO DO

If a learned tracker is built: supervised loss on true target position and velocity, sequence length 128 to 256, gradient clipping 0.5, learning rate 1e-4. Freeze the tracker while the walker is first fine-tuned, then unfreeze lightly.

Question 11

Why did full recurrence underperform here? Which failure mode is most likely: optimization instability, excessive capacity, full-policy relearning, reward competition, insufficient sequence training, lack of explicit search state, or something else?

SHORT ANSWER

Most likely, in order: full-policy relearning from scratch, then capacity and instability together, then the missing explicit search state. Reward competition was a real but PPO_31-only factor. Insufficient sequence training is the least likely.

The relearning explanation fits every data point: PPO_19 to 21 (memory brain from scratch on the easy task) tied or lost to the warm-started plain brain; PPO_30 and 31 (from scratch on the hard task) lost badly; every warm-started plain brain held its ground. Capacity and instability show in the oscillating checkpoints and in PPO_30's final model being worse than its own 200k version. The missing search state explains why even a well-trained notepad would have struggled: it had to invent a map from hundreds of steps of rays without ever being rewarded for the map itself.

Sequence training is unlikely to be the culprit because the same setup produced a decent 71% in PPO_19, where the task did not need long memory. If sequence handling were broken, that run would have failed too.

Question 12

How should PPO_31 be interpreted? It changed memory depth, exploration rewards, and PPO epochs at once and stopped at 163,840 without a recorded exception. What clean ablations should replace it?

SHORT ANSWER

Interpret PPO_31 as 'inconclusive, do not repeat'. It shows that a large exploration bonus creates a wanderer, and that the two-layer brain did not learn faster. It cannot say which change did the damage. The only cheap ablation worth running is the exploration bonus alone on the plain brain; do not spend another 90-minute memory-brain run on it.

The stop should be recorded as 'process ended, cause unknown' with the checkpoint list and the last TensorBoard step, which is how the dossier already words it. Evaluating the 160k checkpoint on the 100-maze exam gives a proper number for the record instead of the 25-maze estimate.

The useful ablation costs ten minutes: PPO_29's exact configuration plus PPO_31's exploration rewards (0.05 per new cell, 0.01 per new view). If success drops and episode length balloons, the wanderer effect is confirmed on its own and the memory brain is exonerated on that count. The two-layer question is not worth answering: even one layer lost.

WHAT TO DO

Phase 0 in Part 6: evaluate the 160k checkpoint, capture the process with stdout and stderr redirected, note free memory during a short re-run, and close the file.

Question 13

How can navigation behaviour be protected during new-task learning? Evaluate staged freezing, behaviour cloning or distillation from PPO_25 or PPO_29, an action-preservation loss, separate learning rates, and hierarchical control.

SHORT ANSWER

Use, in this order: warm start with zeroed new columns and a fresh optimizer (proven here, free); a body-health check at every checkpoint (free); staged freezing if the check trips (cheap); distillation or an action-preservation loss only if freezing is not enough (needs custom code); hierarchical control as the long-term structure for the moving target.

Warm start + zero columns + optimizer reset. This has carried the walker through PPO_25 to 27 to 28 to 29 without visible loss. It should be the default and the first thing tried. Add a modest learning rate ($1.5e-4$) for the first run to be safe.

Staged freezing. Set `requires_grad` to False on the old weights for the first 50k steps, train only the new input columns and the output heads, then unfreeze. Simple in PyTorch. Costs a little sample efficiency. Use it if the body-health check shows drift.

Distillation / action-preservation. Add a penalty for deviating from PPO_29's action on states where the target is visible and the map is irrelevant. Effective but it means a custom PPO loss in Stable-Baselines3, a real engineering task. Keep it in reserve.

Separate learning rates. Parameter groups with a lower rate for old weights. Minor benefit, minor effort; a fine companion to freezing.

Hierarchical control. The walker literally cannot be damaged if it is a frozen module that a planner feeds waypoints to. Highest protection, highest build cost, and the walker must

first be validated on arbitrary waypoints (PPO_25 was only ever trained toward one fixed target, so it is a donor, not a proven waypoint follower). Reserve for the moving-target stage.

WHAT TO DO

Define body health now: collision steps under 6%, stopped steps under 10%, success-after-first-sight above 90%. Print all three at every checkpoint. Any run that breaches them triggers freezing on the next attempt.

Question 14

Should the critic be recurrent even if the actor's navigation path is feed-forward? Conversely, can a target tracker be shared while actor and critic heads remain task-specific?

SHORT ANSWER

The critic does not need memory if it is given the notes; better still, give it privileged information during training. Yes, share the tracker; keep the heads separate.

The critic exists only to steady training. It is never shipped in the game. So the honest-senses rule does not apply to it: it may see the true target position, the true velocity, the full cleared map, and anything else the simulator knows. A critic that knows the answer gives cleaner scores to an actor that does not, which is exactly what you want under hidden information.

Making the critic recurrent instead is a weaker version of the same idea (it has to guess what the privileged critic is simply told), and in the off-the-shelf policy it doubled the notepad count. Skip it.

Sharing the tracker is right because it is a fact about the world ('where is the target probably'), not about the job. The heads are job-specific: one turns the facts into movement, the other into a score.

WHAT TO DO

In Stable-Baselines3 an asymmetric critic needs a custom policy whose value network reads extra entries that the actor ignores. Start Phase 1 without it; add it as the first engineering task if the checkpoints look noisy.

Question 15

What is the best search-memory representation? An egocentric visited-cell map, frontier direction, a compact coverage vector, a recurrent hidden state, or an external map for the current 62.5-unit arena?

SHORT ANSWER

For this arena: an explicit cleared map kept by the engine, summarised to the brain as 16 Leaper-centred 'distance to unchecked ground' rays. That is a frontier representation in the same coordinate frame as her eyes. The full grid is the fallback if the summary loses information; a recurrent hidden state is the last resort.

The arena is 21 by 21 cells of 3 units. Keeping the grid is free. The question is only how to show it to a small brain. Sixteen rays aligned with the vision rays are compact, rotate with her, and directly answer the question the walker needs answered: 'which way is unexplored, and how far'. A single 'frontier direction' is even smaller but loses the trade-off between near and far frontiers. The full grid as 441 inputs is honest but forces the brain to learn spatial reasoning it does not need.

Because the map is marked 'cleared' using the same visibility rule as the target sensor, it never leaks the target and it naturally accounts for rocks: the shadow behind a rock stays unchecked until she walks around it.

WHAT TO DO

Implement clear-marking in the environment's step function, reusing the existing visibility test; add the 16 rays as observation indices 26-41; draw the map in the training viewer so the owner can see her heading for white space.

Question 16

How should the moving target be represented? Should the controller pursue the predicted current position, an intercept point, or a high-level waypoint? How should uncertainty change behaviour when the target stays hidden?

SHORT ANSWER

Start with the predicted current position, shown alongside the raw last-seen position and a confidence value. Add an intercept direction as one more input once the target is fast enough for it to matter. Let confidence fade to zero on long occlusions so the note flips back to 'not found' and the search behaviour returns.

A slow target (say a quarter of Leaper's speed) barely needs interception; chasing the predicted position catches it. At half speed or more, a lead angle helps, and it is one extra unit vector to compute. Give both and let training pick.

Uncertainty should be a number the walker can read (confidence), not a rule you hard-code into her movement. She will learn that a low-confidence prediction is worth a look but not a commitment. The one hard rule is the reset: below a confidence threshold the target channels revert to the 'never seen' values, which is the state PPO_29 already knows how to search from.

Waypoints belong to the hierarchical version and are not needed for the first moving stages.

WHAT TO DO

Target speed for the first moving stage: 0.25 of Leaper's speed, wandering (not fleeing), direction changes only when visible or every 200+ steps. Fleeing is a large difficulty jump; leave it for later.

Question 17

How should rewards change for moving pursuit without creating exploits? Address first detection, reacquisition, progress caused by Leaper versus the target's own motion, exploration bonuses, time cost, belief accuracy, and interception reward.

SHORT ANSWER

Keep the reward almost exactly as PPO_29, with one essential change: measure progress against where the target was at the previous step, so the target's own motion can neither pay nor punish Leaper. Pay first detection once, never pay reacquisition, keep exploration bonuses only in 'not found' mode, do not reward belief accuracy in the RL loop, and keep +25 for interception.

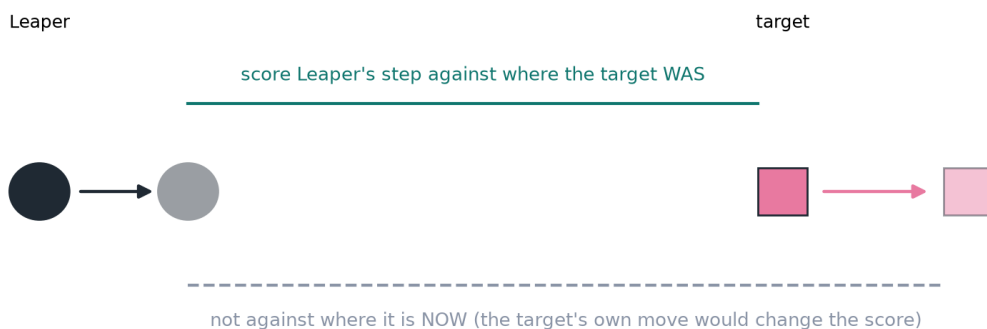
Progress: today's reward is 0.2 times (previous distance minus current distance). If the target walks toward Leaper, that pays her for standing still; if it walks away, it punishes her for chasing. Computing both distances against the target's previous position removes the leak. A target that moves creates no reward on its own.

Reacquisition: PPO_27 proved that paying for re-sightings buys circling. Zero it.

Exploration bonuses: on only while the note says 'not found', including after confidence has fully decayed. Keep them small (0.01); PPO_31 showed what large ones buy.

Belief accuracy: do not pay the walker for it. If a learned tracker exists it is trained by its own supervised loss; if the tracker is explicit it has no knobs. Mixing tracking accuracy into the walker's reward invites her to 'stare at the target' for points.

Time cost and terminal rules stay as they are: -0.002 per step, freeze and stuck rules unchanged. Interception is the goal reward, unchanged at +25.



The one change that keeps the moving-target reward honest.

Question 18

What evaluation protocol would show that memory genuinely helps? Propose fixed seeds, multiple training seeds, target-motion distributions, occlusion-duration buckets, and statistical comparisons against PPO_29 and explicit-state baselines.

SHORT ANSWER

Keep the 100-maze exam (seeds 10,000 to 10,099) as the score of record, add a 1,000-maze confirmation for any new champion, train three seeds per variant, report the mean and spread, and break results down by 'never saw it', 'saw and lost it', 'saw and reached it'. For moving targets, add buckets by target speed and by longest occlusion.

The single most useful new number is **first-detection rate**: the share of episodes where the target was ever seen. It splits the 31% timeouts into 'search failed' versus 'pursuit failed', and it can be computed today for PPO_29 from its replay data before any new training. Add **time to first detection**, **success given detection**, and **fraction of the arena cleared before detection**.

Statistics without ceremony: with three seeds per variant, compare the two means and treat a difference as real when it exceeds the noise band (about nine points on 100 episodes) or when all three seeds move the same way. For a final champion, the 1,000-maze run gives a band of about plus or minus three points.

For moving targets: speeds at 0, 0.25 and 0.5 of Leaper's; occlusion buckets of 0, 1-30, 31-120 and over 120 steps; report success and tracking error (distance between the note's prediction and the truth) per bucket. A memory variant 'helps' when success in the long buckets rises without loss in the short ones.

WHAT TO DO

Add first-detection rate and success-given-detection to final_evaluation.json now. Run them on PPO_29 as the baseline row.

Question 19

Should the project abandon full-policy memory for the static-target stage but keep memory for moving targets? If so, what evidence threshold should trigger adding it?

SHORT ANSWER

Abandon the full-policy memory brain entirely, for both stages. Keep the two-branch brain (Part 4) in reserve for the moving target, and add it only when the arithmetic tracker measurably fails.

The trigger is a number, not a feeling: with the explicit tracker in place, if success-given-detection in the long-occlusion bucket (over 120 steps) is more than 15 points below the short-occlusion bucket, and the tracking error in that bucket is large (the prediction is off by more than the target's radius plus a few units), then prediction is the bottleneck and a learned tracker is justified. If success drops but the tracking error is small, the walker is the problem, not memory.

That test keeps the project from adding recurrence because the target moves, which is the temptation the dossier rightly warns about.

Question 20

What is the smallest next experiment that most reduces uncertainty? Give one architecture, one controlled comparison, a budget, and a go / no-go criterion.

SHORT ANSWER

PPO_32: the PPO_29 brain plus the cleared map as 16 extra inputs, warm-started from PPO_29, three seeds at 250k steps each. Compare against PPO_29 on the 100-maze exam and on first-detection rate. Go if the three-seed mean is 78% or higher with first-detection at 90% or higher. No-go if the mean is 73% or lower.

Why this one: it tests the diagnosed blocker directly, it is the cheapest possible run (about ten minutes per seed on the plain brain), it cannot damage the walker at the start, and both outcomes teach something. A win means memory of coverage was the missing piece and the same mechanism extends to the moving target. A loss means search failures are not about coverage, which redirects attention to sensing range, rock shadows, or the search reward itself.

Between 73% and 78% is the grey zone: extend the best seed to 500k and re-examine before deciding.

Keep everything else identical to PPO_29: arena, speeds, rewards, freeze rule, scan nudge, learning rate, batch settings. One change.

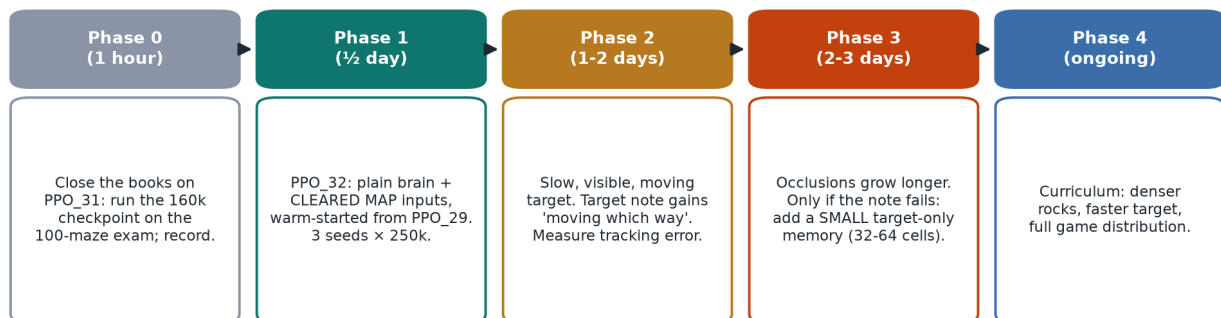
WHAT TO DO

Exact recipe: observation 26 to 42; new columns zero-initialised in both actor and critic first layers; optimizer reset; learning rate $1.5e-4$; 250k steps; seeds natural, three of them; evaluation on the 100-maze exam plus first-detection rate. Artifact directory `rl_artifacts/ppo_32_cleared_map_250k_s1..s3`. Tests: map never marks an unseen cell, map never depends on the target, indices 0-25 unchanged, transferred weights reproduce PPO_29's actions on 1,000 sampled states when the new inputs are zero.

Part 6. The path ahead

Gate between phases: the 100-maze deterministic exam, 3 training seeds, and a written go / no-go number.

Fallback at every step: PPO_29 (69%) stays the champion until something beats it by more than the noise band.



Phases, rough effort, and the gate between them.

Phase 0. Close the PPO_31 file (about an hour)

- Run the 160k checkpoint through the 100-maze exam and write the number into TRAINING.md. This turns 'about 20%' into a real figure.
- Record the stop as 'ended externally, cause not captured'. If curiosity remains, re-launch for 20k steps with stdout and stderr redirected to files and a memory monitor running, then stop. Do not resume PPO_31 for real.
- Optional ten-minute ablation: PPO_29's configuration plus the five-times exploration bonus, to confirm the wanderer effect on its own.

Phase 1. PPO_32, the cleared-map seeker (about half a day)

- Environment: mark cells cleared using the target-visibility rule; compute the 16 'distance to unchecked ground' rays; append them as indices 26-41; draw the map in the viewer.
- Trainer: extend the warm-transfer path so a 26-input donor can load into a 42-input model with zeroed new columns.
- Diagnostics: add first-detection rate, time to first detection, success given detection, arena fraction cleared before detection.
- Run three seeds at 250k. Gate: mean of the three at 78% or higher with first-detection at 90% or higher. Then a 1,000-maze confirmation for the new champion.
- If it fails: look at the failing episodes in the viewer before designing anything else. Where was the target? Was it within 28 units at some point but behind a rock? Did she clear the map and still miss it? The answer picks the next experiment.

Phase 2. A slow, visible, moving target (one to two days)

- Target wanders at a quarter of Leaper's speed, changes direction rarely, does not flee.
- Target note gains velocity, confidence, observed-or-predicted flag, predicted direction and distance (indices 42-48).

- Reward: progress measured against the target's previous position; no reacquisition reward; exploration only in 'not found' mode.
- Warm-start from PPO_32. Report success and tracking error by target speed. Gate: success within the noise band of PPO_32 at speed 0 (no regression) and above 60% at 0.25 speed.

Phase 3. Occlusions and, only if needed, a small learned tracker (two to three days)

- Grow occlusion length through the buckets 1-30, 31-120, over 120 steps by adding rocks near the target's path.
- Apply the Question 19 trigger. If prediction is the bottleneck, build the two-branch brain from Part 4: a 32 to 64-cell LSTM branch fed only the target stream, taught by the simulator's true target position, joined to the walker at zero. Compare it against the arithmetic note on identical seeds and budget.
- The walker is never replaced. If the flat walker stalls on long occlusions even with a good tracker, that is the moment for a planner that hands it waypoints.

Phase 4. Toward the game (ongoing)

- Curriculum: denser rocks, faster target, direction changes while hidden, then the full game distribution.
- Advance a stage only when the deterministic exam is stable, never on stochastic training success.
- Export the champion to the browser with the same care as PPO_25: the JavaScript port must compute the cleared map and target note identically, and the fixture self-test must be regenerated.

Habits that make all of this work

- One change per run. PPO_31 is the cautionary tale.
- Three seeds before believing anything.
- The 100-maze exam is the only score of record. Spot checks and stochastic success are for watching, not deciding.
- Every run keeps the walker: warm start, zeroed new columns, body-health printed at every checkpoint.
- Write the pass mark down before the run starts.

Part 7. One-page cheat sheet

Question in short	Answer in short
Why did the memory brain lose?	It replaced the whole walker with a brain 52 to 140 times bigger, trained from zero, with the rays routed through the memory. Not because memory is a bad idea.
Can the walker be kept?	Yes. Append new senses, zero the new columns, reset the optimizer, warm-start from PPO_29. Never replace the brain.
Memory inside or outside the brain?	Outside first: notes written by the game (cleared map, target note). Inside only later, small, target-only, and only if a measured test demands it.
The two-branch brain?	Good idea, Phase 3: plain walker branch + a 32-64 cell LSTM branch fed only the target stream, joined at zero, taught by the true target position. Race it against the arithmetic note.
What goes into memory?	Target sightings, Leaper's own movement, time since sighting, position history. Not the rays.
Which donor?	PPO_29. PPO_25 only if a separate waypoint walker is ever built.
Two heads?	No. Tracker + cleared map + walker. The critic may see privileged information during training.
Search-memory format?	Engine-kept cleared map, shown as 16 'distance to unchecked ground' rays aligned with the vision rays.
Moving-target reward?	Score progress against where the target was last step. Pay first sight once. Never pay re-sighting.
Evaluation?	100-maze exam, three seeds, first-detection rate and success-given-detection, then 1,000 mazes for a champion.
Next experiment?	PPO_32 = PPO_29 + cleared map, 3 seeds × 250k. Go at 78%+ mean and 90%+ first detection. No-go at 73% or below.
When to add a learned memory?	When the explicit tracker's long-occlusion success is 15+ points below short-occlusion and its prediction error is large.
PPO_31?	Inconclusive, three changes at once, stopped for an unrecorded reason. Evaluate its 160k checkpoint for the record and move on.

Numbers worth remembering

Item	Value
Noise band of a 100-episode exam near 70%	about plus or minus 9 points
One episode in a 25-maze spot check	4 points
PPO_29 (seeker champion)	69% success, 6.3% stopped steps, 4.8% collision steps, 170 steps per episode
PPO_30 memory brain	53% at 200k, 39% at 500k, 623,045 knobs
PPO_31 memory brain	about 20% at 160k (25 mazes), 1,675,717 knobs
Plain brain training speed	about 2 seconds per 1,000 steps (250k in about 9 minutes)

Item	Value
Memory brain training speed	about 15 seconds per 1,000 steps (500k in about 95 minutes)
Sight cone	270 degrees, 28 units, 16 rays; target reported only in view, in range, unblocked
Episode budget	1,000 steps at 0.375 units = 375 units of travel, six arena widths

Where this came from

TRAINING.md (the run ledger through PPO_31 and the planned coverage-map direction), AGENTS.md (current handoff), RL_SPEC.md (exact observation, action, reward, and constants), rl_environment.py (visibility test, target memory, freeze and stuck rules, reward), train_rl.py (plain and recurrent model construction, warm transfer, evaluation loops), PPO_19_memory_report.md and RUN_LOG files (earlier memory experiments), and the question dossier. No new training was run for this document; every number is from the recorded artifacts as reported in those files.